Cryptography

Public-Key Cryptography and RSA

Ksawery Możdżyński

Agenda For Today

1. One shared key
What it cannot do

2. A pair of keys
A padlock anyone can close

3. Hard problems
Easy one way, hard the other

4. Arithmetic modulo n
Euclid and the inverse

5. RSA: keys and encryption
From two primes to a message

6. Digital signatures
Only you could have signed it

7. RSA in the real world
Padding, key sizes, where it lives

Today you will need



version 3.8 or newer



any one you like

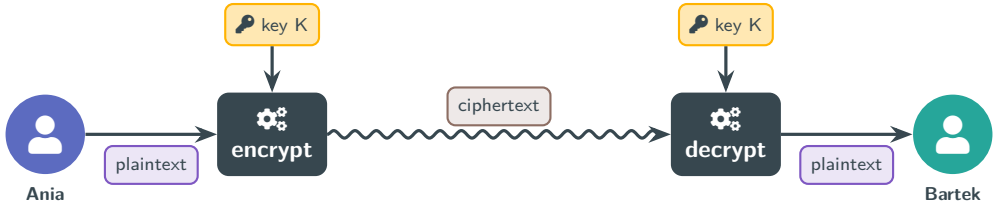


Task 1 and Task 4

Starter files for the tasks: **GitHub**, folders Task2 to Task8. Nothing to install.

Recall: a symmetric cipher

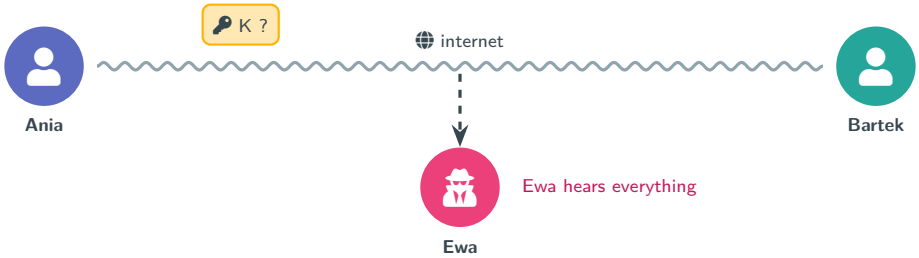
Symmetric cipher: the same secret key encrypts and decrypts.
Examples: RC4, ChaCha20, AES.



Both sides must already have the same key.

Problem 1: how do we share the key?

Ania and Bartek have never met. The internet is all they have.

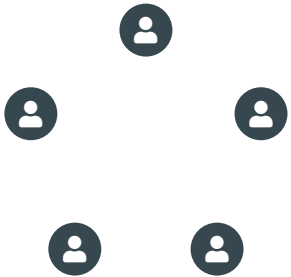


If Ania sends the key over the internet, Ewa has it too.

To talk secretly, we first need a secret.

Task 1: how many keys?

Task 1
On paper, about 3 minutes.



Every pair of people needs its own key: nobody else may read their messages.

How many keys do we need for

- 5 people?
- 30 people (this room)?
- 10 000 people (a company)?

Solution: how many keys?



5 people, 10 keys

n people, each pair one key: $\frac{n(n-1)}{2}$

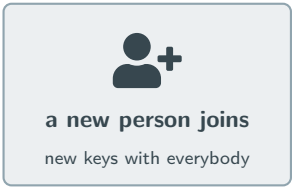
people	keys
5	10
30	435
10 000	49 995 000

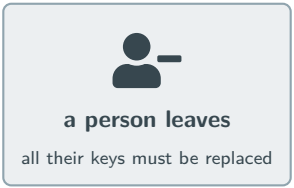
Each person has $n-1$ partners, and each key is shared by two people.

The number of keys grows with the square of the number of people.

Problem 2: keeping track of keys

Each person stores a key for every partner.

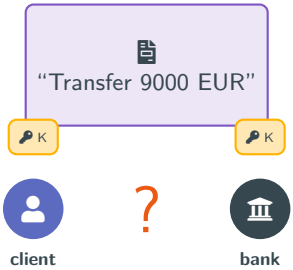






Problem 3: who sent it?

Non-repudiation: the sender cannot later deny that they sent the message.



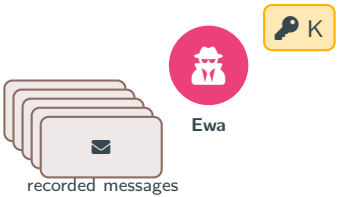
Ania and Bartek share key K. A message encrypted with K could come from either of them.

The client says:
"I never sent it."

The bank has the same key:
so it could have written the order itself.

A shared key cannot prove who wrote a message.

Problem 4: one key leaks



Ewa steals the key K of Ania and Bartek:

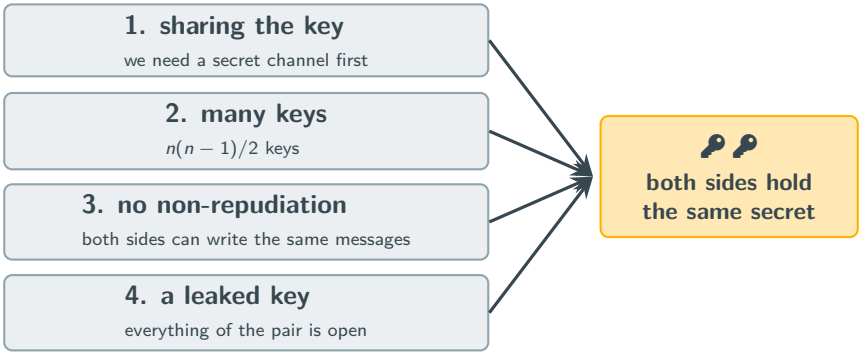
the past
she reads all their old messages that she recorded

the future
she reads all their new messages

impersonation
she can write as Ania to Bartek, and as Bartek to Ania

One stolen key opens everything the pair ever said.

Four problems, one cause



What if the key for locking were public?

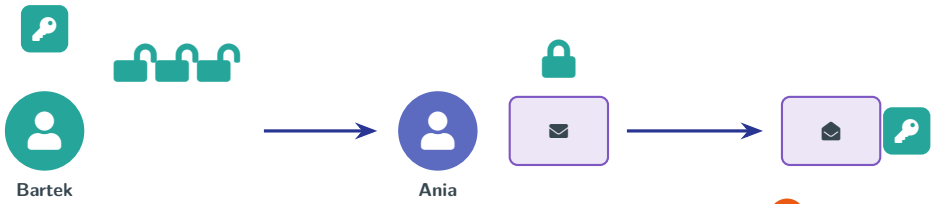


Anybody may lock.



Only one person can unlock.

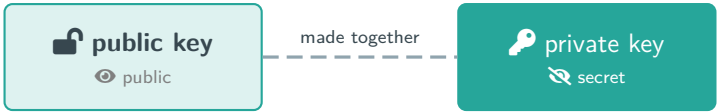
A padlock anyone can close



Closing is public. Opening is private.

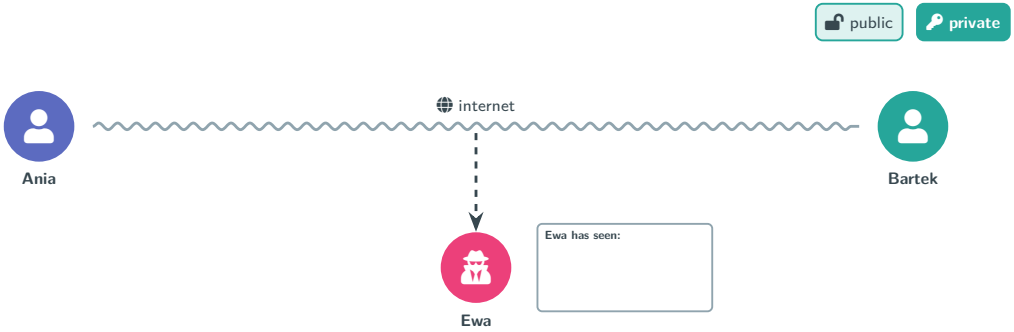
A key pair

Public key: locks (encrypts). Everybody may know it.
Private key: unlocks (decrypts). Only its owner knows it.
They are made together, and the private key cannot be computed from the public one.



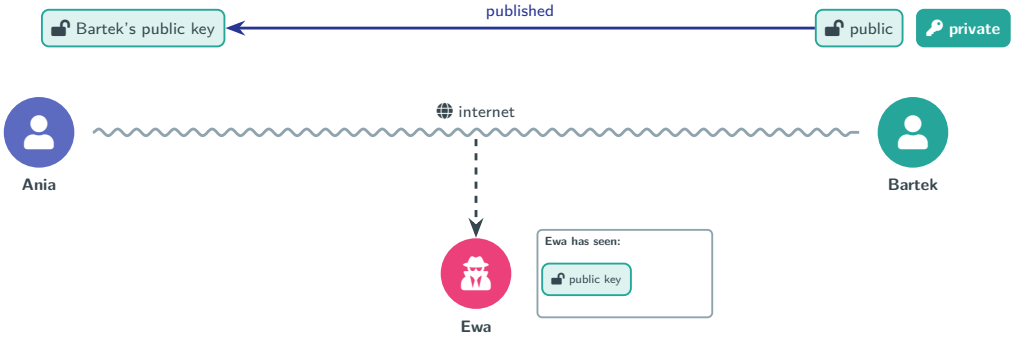
This is **public-key** (asymmetric) cryptography.

The flow, step 1



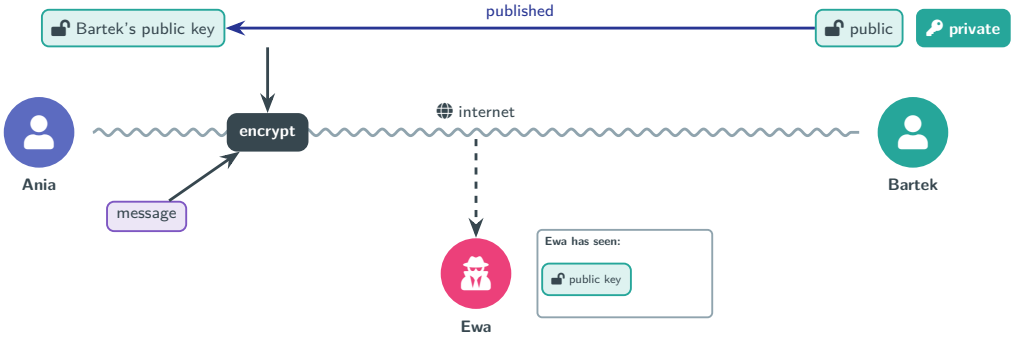
1 Bartek makes a key pair. step 1 of 5

The flow, step 2



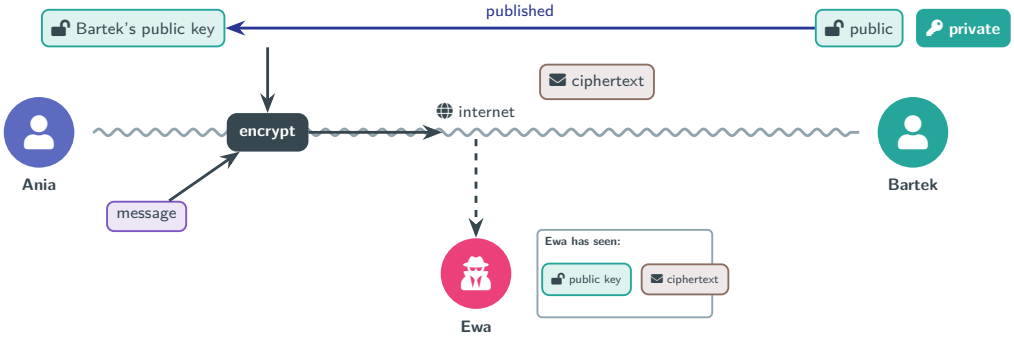
2 Bartek publishes the public key. step 2 of 5

The flow, step 3



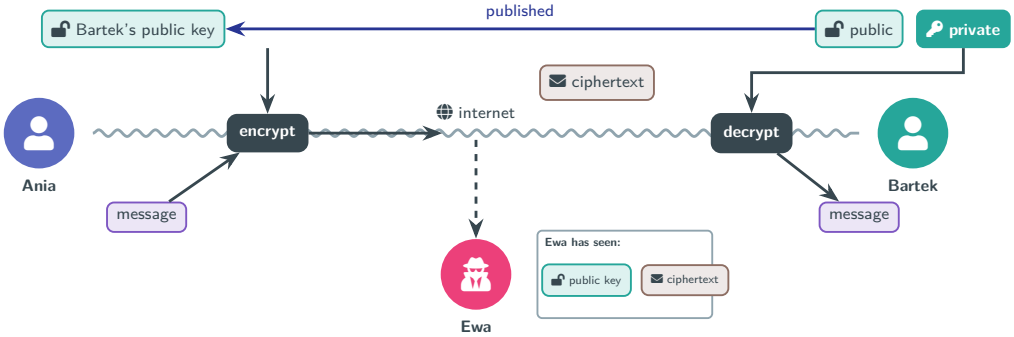
3 Ania encrypts with Bartek's public key. step 3 of 5

The flow, step 4



4 Ania sends the ciphertext. Ewa sees it. step 4 of 5

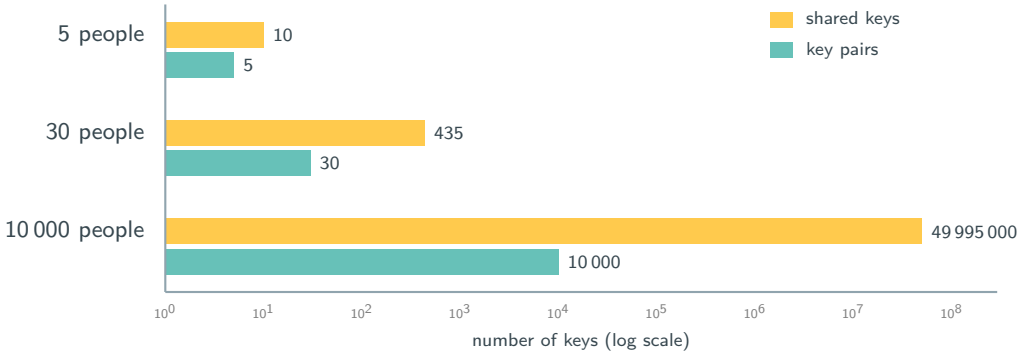
The flow, step 5



5 Bartek decrypts with his private key. step 5 of 5

Ewa saw the public key and the ciphertext. Neither helps her.

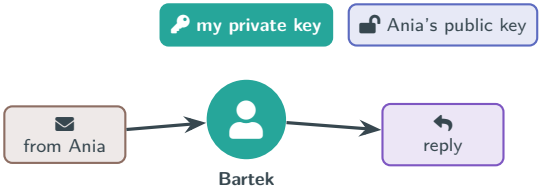
How many keys now?



One key pair per person, however many partners they have.

Task 2: use RSA

Task 2
RSA is the best known public-key cipher. Use it as a black box first.



You are Bartek. You have your private key, Ania's public key and a message from Ania.

1. Decrypt Ania's message.
2. Encrypt a reply for Ania.
3. Try to decrypt your own reply. What happens?

Task 2: what to do

```
from rsa_ready import encrypt, decrypt

my_private_key = (d, n)    # 617-digit numbers
ania_public_key = (65537, n)
ciphertext = ...           # from Ania

# TODO 1: decrypt the ciphertext and print it
# TODO 2: encrypt "Yes, at 5 pm." for Ania
# TODO 3: decrypt your reply with YOUR private key
```

Task 2

Open use_rsa.py from **GitHub**.
rsa_ready.py is a black box.

encrypt(text, public_key)
gives a number

decrypt(number, private_key)
gives the text

Solution: use RSA

```
print(decrypt(ciphertext, my_private_key))
reply = encrypt("Yes, at 5 pm.", ania_public_key)
print(decrypt(reply, my_private_key))
```

The output

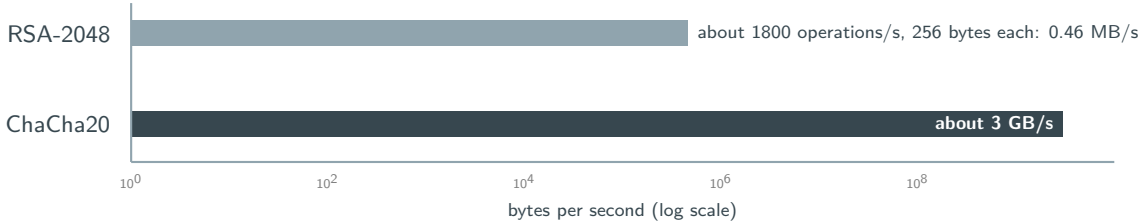
```
Can we meet on Friday at the
library?
```

⚠ Your own reply

`'\x06\xe5Z\x9c...'` (nonsense)

What is locked with Ania's public key, only Ania's private key opens.

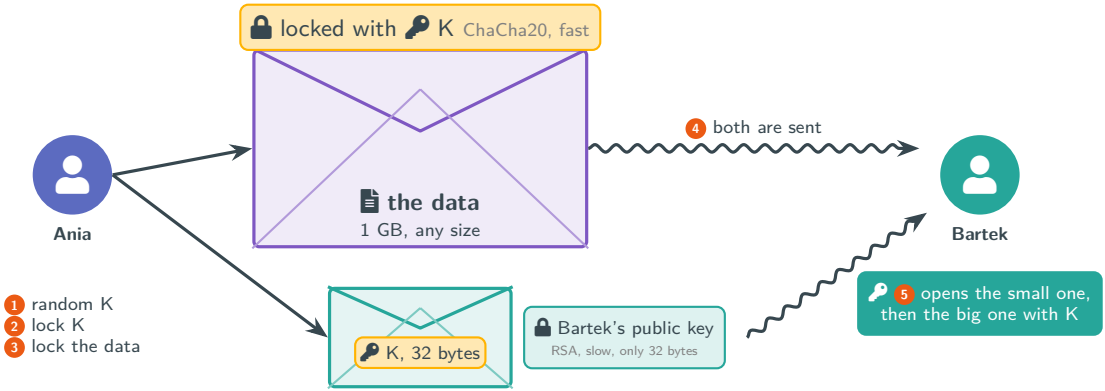
How fast is it?



Encrypting a 1 GB file piece by piece with RSA would take **more than half an hour**.

Public-key crypto is far too slow for bulk data.

Hybrid encryption

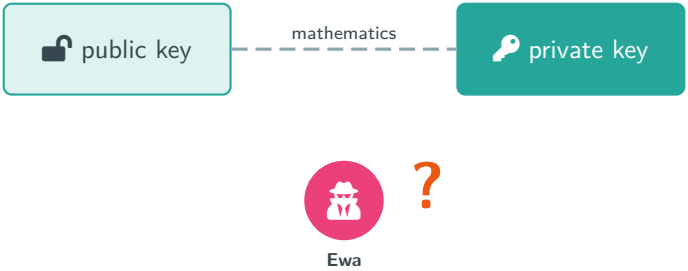


Public-key crypto moves the key, symmetric crypto moves the data.

Which problems are solved?

	problem	with a key pair
✓	1. sharing the key	solved: the public key may travel openly
✓	2. many keys	solved: one pair per person
🕒	3. no non-repudiation	solved later today, in section 6
◐	4. a leaked key	a stolen private key hurts one person, not a pair

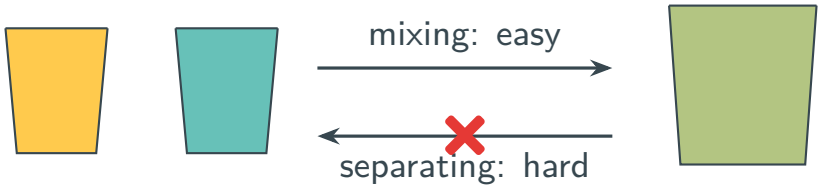
But why can nobody compute the private key?



The two keys are made together, so they must be connected by mathematics.
Why can Ewa not run that mathematics backwards?

Easy one way, hard the other

Trapdoor one-way function: easy to compute, practically impossible to reverse, unless you know a secret (the **trapdoor**).



Multiplying is easy

$$35\,573 \times 38\,839 = 1\,381\,619\,747$$



Python does it in less than a microsecond,
and in a few microseconds for numbers with 300 digits.

Factoring is hard

$$1\,381\,619\,747 = ? \times ?$$

The simple way: try every divisor 2, 3, 5, 7, ... until one fits.

Breaking a number into primes is called **factoring**.

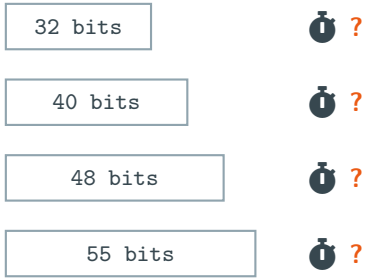
Multiplying is easy. Going back is hard.

Task 3: factor it

Task 3
Four numbers, each a product of two primes: 32, 40, 48 and 55 bits.

1. Write the trial division.
2. Measure the time for each number.

Predict first
Each number is 8 bits longer. How many times longer will it take?



Task 3: what to do

```
def smallest_factor(n):
    # TODO: try the divisors 2, 3, 5, 7, 9, ...
    #       until one divides n (n % f == 0),
    #       and return it.
    #       You only need to try f while
    #       f * f <= n. Why?
    return None
```

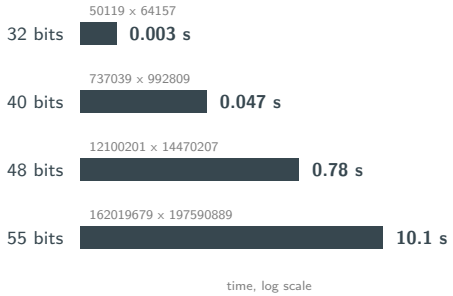
Task 3

Open factor.py from **GitHub**. Write the function in place of the TODO.

The numbers and the time measurement are ready in the file.

Solution: factor it

```
if n % 2 == 0:
    return 2
f = 3
while f * f <= n:
    if n % f == 0:
        return f
    f += 2
```



Every 8 more bits make factoring about 16 times slower.

And 2048 bits?

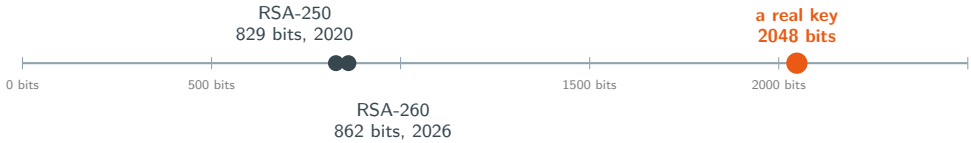
An RSA number has **2048 bits**. Its factors have 1024 bits each.



More than the number of atoms in the universe, many times over.

Better methods exist

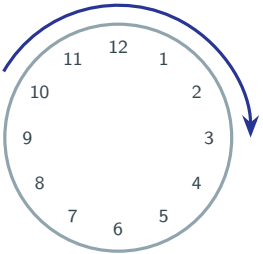
The best known method is the **number field sieve**. It is much faster, but still not enough.



RSA-250: about 2700 years of one processor core. RSA-260 is the current public record.

RSA is safe because nobody can factor 2048-bit numbers.

Recall: mod



10 o'clock + 5 hours

a mod n
the remainder after dividing a by n

examples
$17 \bmod 5 = 2 \qquad (10 + 5) \bmod 12 = 3$

in Python
`17 % 5`

Another hard problem: the discrete logarithm

$5^6 \bmod 23 = 8$ — powers modulo a prime: easy

$5^x \bmod 23 = 8, \quad x = ?$ — going back: hard

For a prime with 2048 bits, nobody can find x in practice.

Four algorithms, two problems

algorithm	year	hard problem	used for
Diffie-Hellman	1976	discrete logarithm	agreeing on a key lecture 4
RSA	1977	**factoring**	encryption, and more today
ElGamal	1985	discrete logarithm	encryption lecture 4
ECC	1985	discrete logarithm on a curve	all of these, with shorter keys lecture 5

Every public-key algorithm stands on one hard problem.

Good to know: GCHQ was first



1973

Clifford Cocks at GCHQ, the British intelligence agency, describes the same idea as RSA.

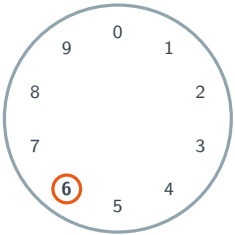
1977

RSA is published by Rivest, Shamir, Adleman.

1997

GCHQ makes it public, twenty years after RSA.

Multiplying modulo n

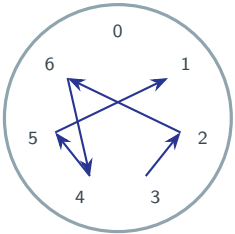


Multiply, then keep the remainder:

$$7 \times 8 \bmod 10 = 56 \bmod 10 = \mathbf{6}$$

The result is always between 0 and n - 1.
Whatever the numbers, it stays smaller than n.

Powers modulo n



3, 2, 6, 4, 5, 1, 3, ...

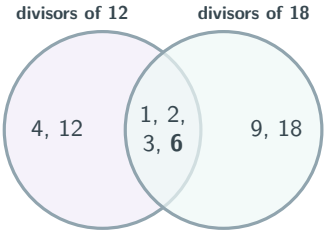
$$3^4 \bmod 7 = 81 \bmod 7 = \mathbf{4}$$

powers of 3 mod 7
3, 2, 6, 4, 5, 1, 3, 2, ...

in Python
`pow(3, 4, 7)` even for numbers with 600 digits

Greatest common divisor

gcd(a, b): the greatest number that divides both a and b.
If gcd(a, b) = 1, a and b are **coprime**.



gcd(12, 18) = **6**

gcd(7, 160) = **1** (coprime)

Euclid's algorithm

$\gcd(a, b) = \gcd(b,\ a \bmod b)$, until the remainder is 0:

a	b	$a = q \times b + r$	r
160	7	$160 = 22 \times 7 + 6$	6
7	6	$7 = 1 \times 6 + 1$	1
6	1	$6 = 6 \times 1 + 0$	0

The last non-zero remainder is the gcd: $\gcd(160, 7) = \mathbf{1}$

The modular inverse

The **inverse of e modulo m**: the number d with $e \times d \bmod m = 1$.
It exists only if gcd(e, m) $= 1$.

the inverse of 7 mod 160 is **23**,

because $7 \times 23 = 161 = 160 + 1$

The inverse undoes a multiplication modulo m.

Why not just try every d?



For m = 160 we can try d = 1, 2, 3, ... and stop at 23.
In RSA, m has about **617 digits**. Trying every d would never end.

We need a shortcut.

Extended Euclid, step 1

a	b	$a = q \times b + r$	r
160	7	$160 = 22 \times 7 + 6$	6
7	6	$7 = 1 \times 6 + 1$	1
6	1	$6 = 6 \times 1 + 0$	0

1. take the row with remainder 1: $1 = 7 - 1 \times 6$

Extended Euclid, step 2

a	b	$a = q \times b + r$	r
160	7	$160 = 22 \times 7 + 6$	6
7	6	$7 = 1 \times 6 + 1$	1
6	1	$6 = 6 \times 1 + 0$	0

1. take the row with remainder 1: $1 = 7 - 1 \times 6$
2. write 6 from the row above: $6 = 160 - 22 \times 7$

Extended Euclid, step 3

a	b	$a = q \times b + r$	r
160	7	$160 = 22 \times 7 + 6$	6
7	6	$7 = 1 \times 6 + 1$	1
6	1	$6 = 6 \times 1 + 0$	0

1. take the row with remainder 1: $1 = 7 - 1 \times 6$
2. write 6 from the row above: $6 = 160 - 22 \times 7$
3. put it in, collect the 7s: $1 = 7 - (160 - 22 \times 7)$

   $1 = \mathbf{23} \times 7 - 1 \times 160$

$23 \times 7 = 1 + 160$, so $23 \times 7 \bmod 160 = 1$: the inverse of 7 is **23**.

The inverse falls out of Euclid's table, read from the bottom.

Task 4: an inverse by hand

Task 4
On paper: find the inverse of 23 modulo 120. About 8 minutes.

1. Euclid's table for gcd(120, 23). It takes 4 rows before the remainder 0.
2. Read it from the bottom, as on the last slides.
3. Check: $23 \times d \bmod 120$ must be 1.

a	b	$a = q \times b + r$
120	23	

Solution: an inverse by hand

a	b	$a = q \times b + r$
120	23	$120 = 5 \times 23 + 5$
23	5	$23 = 4 \times 5 + 3$
5	3	$5 = 1 \times 3 + 2$
3	2	$3 = 1 \times 2 + 1$
2	1	$2 = 2 \times 1 + 0$

$$
\begin{aligned}
1 &= 3 - 2 = 3 - (5 - 3) = 2 \times 3 - 5 \\
&= 2 \times (23 - 4 \times 5) - 5 = 2 \times 23 - 9 \times 5 \\
&= 2 \times 23 - 9 \times (120 - 5 \times 23) = \mathbf{47} \times 23 - 9 \times 120
\end{aligned}
$$

d = 47
$23 \times 47 = 1081 = 9 \times 120 + 1$

In Python
`pow(23, -1, 120)` gives 47

Good to know: fast powers



$3^{16} = ((((3^2)^2)^2)^2)^2)$: **4 squarings**, not 15 multiplications.

A 2048-bit exponent: about 2048 squarings, not 2^{2048} multiplications.

This is why `pow(m, e, n)` is fast.

Making the keys, step 1

	step	example	
1	choose two primes p and q	$p = 17$, $q = 11$	secret
2			
3			
4			
5			

Making the keys, step 2

	step	example	
1	choose two primes p and q	$p = 17$, $q = 11$	secret
2	$n = p \times q$	$n = 17 \times 11 = 187$	public
3			
4			
5			

Making the keys, step 3

	step	example	
1	choose two primes p and q	$p = 17$, $q = 11$	secret
2	$n = p \times q$	$n = 17 \times 11 = 187$	public
3	$phi = (p - 1)(q - 1)$	$phi = 16 \times 10 = 160$	secret
4			
5			

Making the keys, step 4

	step	example	
1	choose two primes p and q	$p = 17$, $q = 11$	secret
2	$n = p \times q$	$n = 17 \times 11 = 187$	public
3	$phi = (p - 1)(q - 1)$	$phi = 16 \times 10 = 160$	secret
4	choose e with $gcd(e, phi) = 1$	$e = 7$, $gcd(7, 160) = 1$	public
5			

Making the keys, step 5

	step	example	
1	choose two primes p and q	$p = 17$, $q = 11$	secret
2	$n = p \times q$	$n = 17 \times 11 = 187$	public
3	$phi = (p - 1)(q - 1)$	$phi = 16 \times 10 = 160$	secret
4	choose e with $gcd(e, phi) = 1$	$e = 7$, $gcd(7, 160) = 1$	public
5	d = the inverse of e mod phi	$d = 23$, $7 \times 23 \bmod 160 = 1$	secret

The two keys



$\{e, n\} = \{7, 187\}$

give it to everybody



$\{d, n\} = \{23, 187\}$

keep it secret



p, q and phi: not needed any more; keep them secret or delete them

Whoever knows p and q can compute d.

Encrypt and decrypt



P must be a number smaller than n.

Encryption and decryption are both one power modulo n.

The example: encrypt 88

$$C = 88^7 \bmod 187$$

$$\begin{aligned}
88^1 \bmod 187 &= 88 \\
88^2 \bmod 187 &= 7744 \bmod 187 = 77 \\
88^4 \bmod 187 &= 77^2 \bmod 187 = 132 \\
88^7 = 88^4 \cdot 88^2 \cdot 88^1 &= 132 \cdot 77 \cdot 88 \bmod 187 = \mathbf{11}
\end{aligned}$$



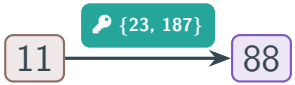
The example: decrypt 11

$$P = 11^{23} \bmod 187, \qquad 23 = 16 + 4 + 2 + 1$$

11^1	11^2	11^4	11^8	11^{16}
11	121	55	33	154

all mod 187

$$11^{23} = 154 \cdot 55 \cdot 121 \cdot 11 \bmod 187 = \mathbf{88}$$

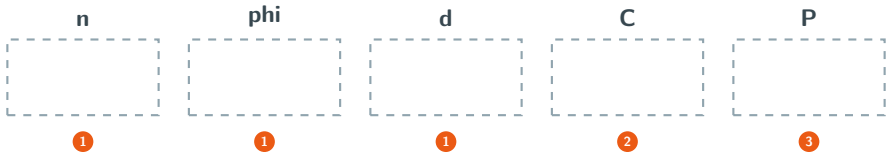


the message is back

The private key undoes what the public key did.

Task 5: your own RSA

Task 5
Two primes with about 20 digits each are given, and e = 65537. About 8 minutes.



1. Compute n, phi and d.
2. Encrypt the number 2026.
3. Decrypt it again.

Task 5: what to do

```
p = 37908704904578413183
q = 58606630333925820283
e = 65537
message = 2026

# TODO 1: n = ...
# TODO 2: phi = ...
# TODO 3: d = ...       pow(e, -1, phi)
# TODO 4: C = ...       pow(message, e, n)
# TODO 5: P = ...
```

Task 5

Open my_rsa.py from **GitHub**. Write the code in place of the TODO lines.

Numbers with 20 digits, so that the result fits on the screen.

Solution: your own RSA

```
n = p * q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
C = pow(message, e, n)
P = pow(C, d, n)
```

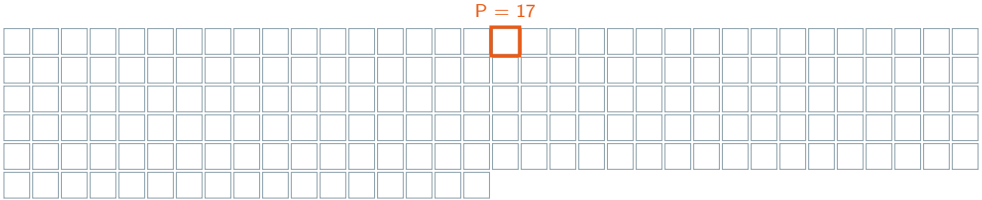
The output

```
d = 136715134915914822141086181985876895038 1
C = 736246060401132335090489988934276767570
P = 2026
```

All of RSA is five lines of Python.

Task 6: does it always work?

Task 6
Key: n = 187, e = 7, d = 23. About 4 minutes.



Encrypt and decrypt every P from 0 to 186. Does every P come back?

Predict first
What about P = 17, which divides n?

Task 6: what to do

```
n, e, d = 187, 7, 23
wrong = []
# TODO: for every P in range(n): encrypt,
#       decrypt, and add P to wrong
#       if it does not come back
print(wrong)
```

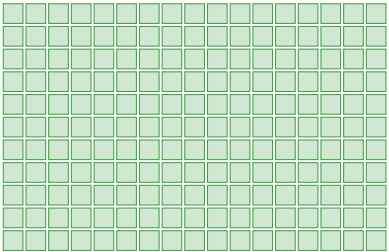
Task 6

Open always.py from **GitHub**.

One loop and one comparison.

Solution: does it always work?

```python
for P in range(n):
    C = pow(P, e, n)
    if pow(C, d, n) != P:
        wrong.append(P)
```



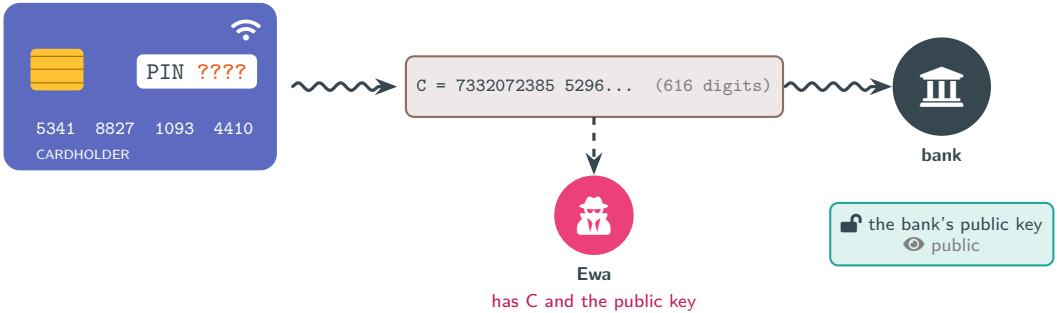
wrong: []

P = 17 -> 85 -> 17

RSA works for every P from 0 to n - 1.

Task 7: guess the PIN

Task 7
A bank encrypts 4-digit PINs with RSA, exactly as on the last slides, with a 2048-bit key. About 7 minutes.



Find the PIN without the private key.

Task 7: what to do

```
e = 65537
n = ...     # the bank's public key, 617 digits
C = ...     # the encrypted PIN

# TODO: find the PIN without the private key.
#       Hint: you can encrypt any number yourself.
```

Task 7

Open pin.py from **GitHub**.

The time measurement is ready in the file.

Solution: guess the PIN

```
for guess in range(10000):
    if pow(guess, e, n) == C:
        pin = guess
        break
```

The output

```
PIN: 4821  found in 0.40 s
```

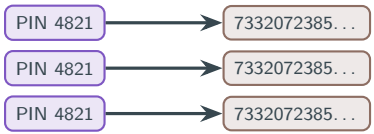
Why it works

Encrypting is public, and there are only 10 000 PINs.

Textbook RSA: the same message always gives the same ciphertext.

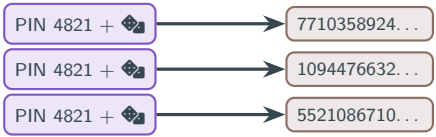
Real RSA adds randomness: OAEP

textbook RSA: $C = P^e \bmod n$



✗ the same every time

real RSA: $C = (P + \textbf{random padding})^e \bmod n$



✓ different every time

The padding is called **OAEP**. The receiver removes it after decrypting.

Never use textbook RSA alone.

Reverse the keys

encryption

$C = P^e \bmod n$

with the public key

$P = C^d \bmod n$

with the private key

signature

$S = M^d \bmod n$

sign with the private key

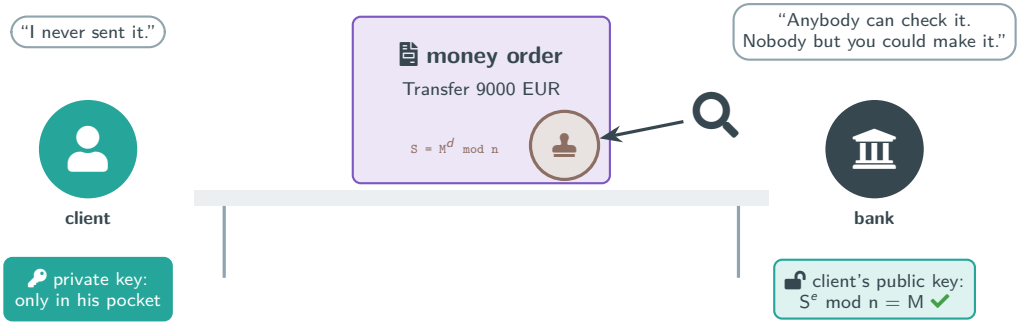
$M = S^e \bmod n$

check with the public key

Only the private key can make a signature. Everybody can check it.

Why this gives non-repudiation

Only the owner of the private key could compute S.



A signature proves who wrote the message.

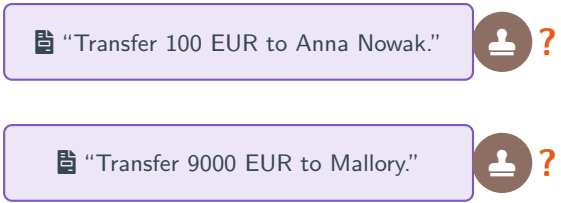
The example: sign 42



Change the message to 43: $168^7 \bmod 187$ is still 42, so the check fails ✗.

Task 8: which one is genuine?

Task 8
Two money orders arrived, each with a signature. About 5 minutes.



You have the bank's public key.

Which order really comes from the bank?

Task 8: what to do

```
e, n = ...      # the bank's public key
orders = [(message, signature),
          (message, signature)]

def verify(message, signature):
    # TODO: True if signature^e mod n
    #       equals the message as a number
    return None
```

Task 8

Open verify.py from **GitHub**.

text_to_number is ready in the file.

Solution: which one is genuine?

```python
def verify(message, signature):
    return pow(signature, e, n) == text_to_number(message)
```

`Transfer 100 EUR to Anna Nowak.`	✔	`genuine: True`
`Transfer 9000 EUR to Mallory.`	✘	`genuine: False`

Without the private key, Mallory cannot make a valid signature.

Real signatures sign a hash



A **hash function** turns a message of any length into a short fingerprint, for example 32 bytes.

The padding for signatures is called **PSS**.

Encrypting and signing side by side

what	who uses which key
encrypt for Bartek	Ania uses Bartek's public key
decrypt	Bartek uses his private key
sign as Ania	Ania uses her private key
verify Ania's signature	anybody uses Ania's public key

Encrypt with their public key, sign with your private key.

How long should the key be?

key size	security	private-key operation (OpenSSL)
2048 bits	112 bits	0.56 ms
3072 bits	128 bits	1.76 ms
4096 bits	about 140 bits	3.76 ms

⚠ NIST: 2048 bits is the minimum, and only until 2030.

Longer keys are safer and slower.

Where RSA lives



TLS (HTTPS)

certificates and server signatures;
TLS 1.3 no longer sends keys with RSA



e-mail

S/MIME and OpenPGP:
encrypted and signed e-mail



VPN and IPsec

the two sides prove who they are
with signatures and certificates



SSH

ssh-rsa keys for logging in
without a password

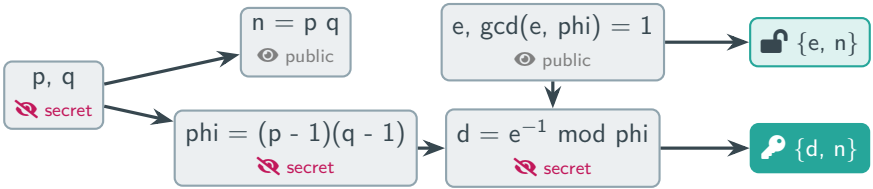
Good to know: quantum computers





Shor's algorithm could factor large numbers fast on a big quantum computer.
No such computer exists yet, but NIST plans to move away from RSA after 2030.

RSA in one picture



encrypt $C = P^e \bmod n$ decrypt $P = C^d \bmod n$

sign $S = M^d \bmod n$ verify $M = S^e \bmod n$

Summary

1. A shared key brings four problems: sharing it, too many keys, no non-repudiation, leaks.
2. A key pair solves sharing; in practice it moves a symmetric key (hybrid encryption).
3. RSA is safe because factoring is hard; DH, ElGamal and ECC use the discrete logarithm.
4. Euclid's algorithm gives the modular inverse d.
5. RSA encrypts and signs with one power mod n; real RSA adds padding (OAEP, PSS).